

microgpt_sft.py — An Annotated Guide

LoRA Supervised Fine-Tuning in Pure Python

@nealrichter + Amazon Kiro

June 2025

Abstract

This document walks through `microgpt_sft.py`, which implements LoRA (Low-Rank Adaptation) supervised fine-tuning on top of a pretrained microgpt model. It demonstrates the complete SFT pipeline: loading a frozen base model, attaching trainable low-rank adapters, training with response-only loss masking, and merging adapters back into the base weights. All in pure Python with zero dependencies.

Source: https://github.com/nealrichter/research/blob/main/microgpt/microgpt_sft.py

Contents

1	Overview	2
2	Model Loading and Vocabulary Extension	2
3	Frozen Base Model	3
4	LoRA Adapters	3
4.1	LoRA Mathematics	3
4.2	Initialization	3
5	LoRA-Augmented Forward Pass	4
6	SFT Data Format	4
7	Response-Only Loss Masking	5
8	Optimizer: Frozen Base + Trainable Adapters	5
9	LoRA Merge and Export	6
10	Usage	6

1 Overview

`microgpt_sft.py` extends `microgpt.py` with post-training fine-tuning:

1. **Load:** Restore pretrained weights from `model.json` (frozen).
2. **Adapt:** Attach LoRA adapters to Q and V attention projections.
3. **Train:** Optimize only adapter parameters on instruction-response pairs.
4. **Merge:** Fold adapters into base weights and save `model_sft.json`.

Parameter	Value
LoRA rank (r)	4
LoRA alpha (α)	8
Target modules	attn_wq, attn_wv
Trainable params	256 (5.7% of base)
W_down init	$\mathcal{N}(0, 0.08^2)$
W_up init	zeros

Table 1: LoRA configuration

2 Model Loading and Vocabulary Extension

The pretrained model is loaded from JSON and extended with a SEP (separator) token to delineate instructions from responses:

```
11 import json, math, random, sys, os
12 random.seed(42)
13
14 # --- Load model ---
15 src_file = model_file if inference_only else 'model.json'
16 with open(src_file) as f:
17     model = json.load(f)
18
19 uchars = model['vocab']
20 cfg = model['config']
21 n_layer, n_embd, n_head = cfg['n_layer'], cfg['n_embd'], cfg['n_head']
22 head_dim = n_embd // n_head
23 BOS = len(uchars)
24 SEP = len(uchars) + 1
25 vocab_size = len(uchars) + 2
```

The vocabulary is extended by one token:

$$V_{\text{sft}} = V_{\text{base}} \cup \{\text{SEP}\}$$

This gives the model a boundary marker between instruction and response in the training format: $[\text{BOS}, \text{instr}_1, \dots, \text{instr}_m, \text{SEP}, \text{resp}_1, \dots, \text{resp}_n, \text{BOS}]$.

3 Frozen Base Model

Base weights are loaded into `Value` nodes but **never updated**—gradients flow through them during backpropagation but are zeroed without applying updates:

```
76 state_dict = {k: [[Value(w) for w in row] for row in mat]
77                  for k, mat in model['weights'].items()}
78
79 # Extend embedding tables for SEP token if needed
80 for key in ('wte', 'lm_head'):
81     if len(state_dict[key]) < vocab_size:
82         state_dict[key].append([Value(random.gauss(0, 0.08))
83                                 for _ in range(n_embd)])
84
85 # Extend positional embeddings for longer sequences
86 block_size = 32
87 while len(state_dict['wpe']) < block_size:
88     state_dict['wpe'].append([Value(random.gauss(0, 0.08))
89                               for _ in range(n_embd)])
89
90
91 base_params = [p for mat in state_dict.values()
92                for row in mat for p in row]
```

4 LoRA Adapters

Low-Rank Adaptation [1] adds a trainable low-rank decomposition to selected weight matrices while keeping the original weights frozen.

4.1 LoRA Mathematics

For a pretrained weight matrix $W_0 \in \mathbb{R}^{d \times d}$, LoRA adds:

$$h = W_0 x + \frac{\alpha}{r} \cdot W_{\text{up}} \cdot W_{\text{down}} \cdot x$$

where:

- $W_{\text{down}} \in \mathbb{R}^{r \times d}$ projects input to low-rank space
- $W_{\text{up}} \in \mathbb{R}^{d \times r}$ projects back to full dimension
- $r = 4$ is the rank (bottleneck dimension)
- $\alpha = 8$ is the scaling factor
- Only W_{up} and W_{down} are trained (256 total parameters)

4.2 Initialization

Per the LoRA paper [1], W_{down} is randomly initialized and W_{up} is initialized to zeros, so the adapter output is zero at the start of training (no perturbation to the pretrained model):

```
92 lora_rank = 4
93 lora_alpha = 8
94 lora_scale = lora_alpha / lora_rank
95
96 matrix = lambda nout, nin, std=0.08: \
97     [[Value(random.gauss(0, std)) for _ in range(nin)]
```

```

98         for _ in range(nout)]
99
100 lora_dict = {}
101 for i in range(n_layer):
102     lora_dict[f'layer{i}.attn_wq.down'] = matrix(lora_rank, n_embd
103         )
104     lora_dict[f'layer{i}.attn_wq.up'] = [[Value(0.0)
105         for _ in range(lora_rank)] for _ in range(n_embd)]
106     lora_dict[f'layer{i}.attn_wv.down'] = matrix(lora_rank, n_embd
107         )
108     lora_dict[f'layer{i}.attn_wv.up'] = [[Value(0.0)
109         for _ in range(lora_rank)] for _ in range(n_embd)]
110
111 lora_params = [p for mat in lora_dict.values()
112     for row in mat for p in row]

```

5 LoRA-Augmented Forward Pass

The key modification is `linear_lora`, which adds the scaled adapter output to the base linear layer:

```

113 def linear(x, w):
114     return [sum(wi * xi for wi, xi in zip(wo, x)) for wo in w]
115
116 def linear_lora(x, w_base, w_down, w_up):
117     """h = W_base @ x + scale * W_up @ (W_down @ x)"""
118     return [b + lora_scale * l
119         for b, l in zip(linear(x, w_base),
120             linear(linear(x, w_down), w_up))]

```

This is applied only to the Q and V projections in attention:

```

128 def gpt(token_id, pos_id, keys, values):
129     x = [t + p for t, p in zip(state_dict['wte'][token_id],
130         state_dict['wpe'][pos_id])]
131
132     x = rmsnorm(x)
133     for li in range(n_layer):
134         x_res = x
135         x = rmsnorm(x)
136         q = linear_lora(x, state_dict[f'layer{li}.attn_wq'],
137             lora_dict[f'layer{li}.attn_wq.down'],
138             lora_dict[f'layer{li}.attn_wq.up'])
139         k = linear(x, state_dict[f'layer{li}.attn_wk'])
140         v = linear_lora(x, state_dict[f'layer{li}.attn_wv'],
141             lora_dict[f'layer{li}.attn_wv.down'],
142             lora_dict[f'layer{li}.attn_wv.up'])

```

The K projection and MLP are **not** adapted—following the finding in [1] that adapting Q and V is sufficient.

6 SFT Data Format

Instruction-response pairs are loaded from `input_sft.txt` in pipe-delimited format:

```

158 sft_data = []
159 for line in open('input_sft.txt'):
160     if '|' in line:

```

```

161         instr, resp = line.strip().split('|', 1)
162         sft_data.append((instr, resp))

```

Example data:

```

fa|Alice
mb|Benjamin

```

The 2-character codes encode gender (f/m) and starting letter.

7 Response-Only Loss Masking

The critical SFT technique: compute cross-entropy loss **only on response tokens**, masking the instruction portion [3, 6]:

```

210         instr_tok = encode(instruction)
211         resp_tok = encode(response)
212         tokens = [BOS] + instr_tok + [SEP] + resp_tok + [BOS]
213         n = min(block_size, len(tokens) - 1)
214         resp_start = len(instr_tok) + 2
215
216         keys, values = [[] for _ in range(n_layer)],
217                        [[] for _ in range(n_layer)]
218         losses = []
219         for pos_id in range(n):
220             logits = gpt(tokens[pos_id], pos_id, keys, values)
221             if pos_id >= resp_start - 1:
222                 probs = softmax(logits)
223                 losses.append(-probs[tokens[pos_id + 1]].log())
224
225         loss = (1 / len(losses)) * sum(losses)

```

The loss function for SFT:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{|R|} \sum_{j \in R} \log P(t_j | t_1, \dots, t_{j-1}; \theta)$$

where R is the set of response token positions (after SEP). Instruction tokens contribute to the forward pass (building context in the KV-cache) but not to the loss gradient.

8 Optimizer: Frozen Base + Trainable Adapters

Only LoRA parameters receive gradient updates. Base model gradients are accumulated during backward but explicitly zeroed:

```

227         lr_t = lr * (1 - step / total_steps)
228         for i, p in enumerate(lora_params):
229             m[i] = beta1 * m[i] + (1 - beta1) * p.grad
230             v_buf[i] = beta2 * v_buf[i] + (1 - beta2) * p.grad **
231                        2
232             p.data -= lr_t * (m[i] / (1 - beta1 ** (step + 1))) \
233                        / ((v_buf[i] / (1 - beta2 ** (step + 1))) ** 0.5 + eps)
234             p.grad = 0
235         for p in base_params:
236             p.grad = 0

```

This is the “parameter-efficient” aspect of LoRA: we train 256 parameters instead of 4,480.

9 LoRA Merge and Export

After training, the adapter weights are merged back into the base model so inference requires no LoRA-specific code:

```
244     for i in range(n_layer):
245         for key in (f'layer{i}.attn_wq', f'layer{i}.attn_wv'):
246             w_base = state_dict[key]
247             w_down = lora_dict[f'{key}.down']
248             w_up = lora_dict[f'{key}.up']
249             for r in range(len(w_base)):
250                 for c in range(len(w_base[0])):
251                     w_base[r][c].data += lora_scale * sum(
252                         w_up[r][k].data * w_down[k][c].data
253                         for k in range(lora_rank))
```

The merge operation:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} \cdot W_{\text{up}} \cdot W_{\text{down}}$$

After merging, the model is saved as `model_sft.json` and can be loaded by `microgpt.py -i model_sft.json` without any adapter code.

10 Usage

```
1  # Step 1: Pretrain base model (required first)
2  python3 microgpt.py
3
4  # Step 2: Train LoRA SFT (loads model.json, saves model_sft.json)
5  python3 microgpt_sft.py
6
7  # Inference only from saved SFT model
8  python3 microgpt_sft.py -i
9  python3 microgpt_sft.py -i model_sft.json
10
11 # Use merged model with base inference script
12 python3 microgpt.py -i model_sft.json
```

References

- [1] Hu, J.E., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., and Chen, W. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685, 2021. <https://arxiv.org/abs/2106.09685>
- [2] Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. *QLoRA: Efficient Finetuning of Quantized LLMs*. Advances in Neural Information Processing Systems 36, 2024. <https://arxiv.org/abs/2305.14314>
- [3] Tie, G., Zhao, Z., Song, D., et al. *A Survey on Post-training of Large Language Models*. arXiv:2503.06072v3, March 2025. <https://arxiv.org/abs/2503.06072>
- [4] Vaswani, A., Shazeer, N., Parmar, N., et al. *Attention Is All You Need*. Advances in Neural Information Processing Systems 30, 2017. <https://arxiv.org/abs/1706.03762>
- [5] Kingma, D.P. and Ba, J. *Adam: A Method for Stochastic Optimization*. Proceedings of ICLR, 2015. <https://arxiv.org/abs/1412.6980>

- [6] Wei, J., Bosma, M., Zhao, V., et al. *Finetuned Language Models Are Zero-Shot Learners*. Proceedings of ICLR, 2022. <https://arxiv.org/abs/2109.01652>